Machine Learning with Big Data

Assignment 3

Recommender Systems using Content-Based and Collaborative Filtering on the MovieLens Dataset

Submitted by - Akshay Kumar (M25DE1028)

GitHub Link – Assignment 3

Introduction

The MovieLens dataset is a widely used benchmark dataset for recommender system research. It contains user ratings for movies along with movie metadata such as title and genres.

For this assignment, the **MovieLens Latest Small dataset** was used. The dataset contains approximately 100,000 ratings from around 600 users on nearly 9,000 movies.

The following files were used:

movies.csv – contains movieId, movie title, and genres
ratings.csv – contains userId, movieId, rating, and timestamp

Genres were treated as textual descriptions of movies and used as input features for building content-based recommendation models.

Step 1: Creating the folder structure

Name	Status	Date modified	Type	Size
dataset		17-03-2026 21:41	File folder	
Assignmnet3_Recommender		17-03-2026 21:47	Jupyter Source File	0 KB
M25DE1028_CSL7110_Assignment3		17-03-2026 21:57	Microsoft Word D...	350 KB
README		17-03-2026 21:43	Markdown Source ...	0 KB
requirements		17-03-2026 21:41	Text Document	0 KB

Step 2: Initialize git and first commit

```
aksha@AkshayKumar MINGW64 ~/OneDrive/Desktop/IITJ/Machine Learning with Big Data/Assignmnet 3/CSL7110_Assignment3_RecommenderSystems (master)
$ git status
fatal: not a git repository (or any of the parent directories): .git

aksha@AkshayKumar MINGW64 ~/OneDrive/Desktop/IITJ/Machine Learning with Big Data/Assignmnet 3/CSL7110_Assignment3_RecommenderSystems
$ git init
Initialized empty Git repository in C:/Users/aksha/OneDrive/Desktop/IITJ/Machine Learning with Big Data/Assignmnet 3/CSL7110_Assignment3_Recommende
Systems/.git/

aksha@AkshayKumar MINGW64 ~/OneDrive/Desktop/IITJ/Machine Learning with Big Data/Assignmnet 3/CSL7110_Assignment3_RecommenderSystems (master)
$ git add .

aksha@AkshayKumar MINGW64 ~/OneDrive/Desktop/IITJ/Machine Learning with Big Data/Assignmnet 3/CSL7110_Assignment3_RecommenderSystems (master)
$ git commit -m "Initial project structure for Assignment 3"
[master (root-commit) 09b35b6] Initial project structure for Assignment 3
 4 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Assignmnet3_Recommender.ipynb
 create mode 100644 M25DE1028_CSL7110_Assignment3.docx
 create mode 100644 README.md
 create mode 100644 requirements.txt
```

Step 3: Connect GitHub

```
aksha@AkshayKumar MINGW64 ~/OneDrive/Desktop/IITJ/Machine Learning with Big Data/Assignmnet 3/CSL7110_Assignment3_RecommenderSystems (master)
$ git branch -M main

aksha@AkshayKumar MINGW64 ~/OneDrive/Desktop/IITJ/Machine Learning with Big Data/Assignmnet 3/CSL7110_Assignment3_RecommenderSystems (main)
$ git remote add origin https://github.com/Akshaykumarky26/CSL7110_Assignment3_RecommenderSystems.git

aksha@AkshayKumar MINGW64 ~/OneDrive/Desktop/IITJ/Machine Learning with Big Data/Assignmnet 3/CSL7110_Assignment3_RecommenderSystems (main)
$ git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 24 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 246.09 KiB | 12.30 MiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Akshaykumarky26/CSL7110_Assignment3_RecommenderSystems.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Step 4: Download Dataset and push

```
aksha@AkshayKumar MINGW64 ~/OneDrive/Desktop/IITJ/Machine Learning with Big Data/Assignmnet 3/CSL7110_Assignment3_RecommenderSystems (main)
$ git add dataset

aksha@AkshayKumar MINGW64 ~/OneDrive/Desktop/IITJ/Machine Learning with Big Data/Assignmnet 3/CSL7110_Assignment3_RecommenderSystems (main)
$ git commit -m "Added MovieLens dataset"
[main d8dadf1] Added MovieLens dataset
 4 files changed, 124007 insertions(+)
 create mode 100644 dataset/links.csv
 create mode 100644 dataset/movies.csv
 create mode 100644 dataset/ratings.csv
 create mode 100644 dataset/tags.csv

aksha@AkshayKumar MINGW64 ~/OneDrive/Desktop/IITJ/Machine Learning with Big Data/Assignmnet 3/CSL7110_Assignment3_RecommenderSystems (main)
$ git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 24 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 950.91 KiB | 7.10 MiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Akshaykumarky26/CSL7110_Assignment3_RecommenderSystems.git
   01b35c3..d8dadf1  main -> main
```

Part 1: Content-Based Filtering

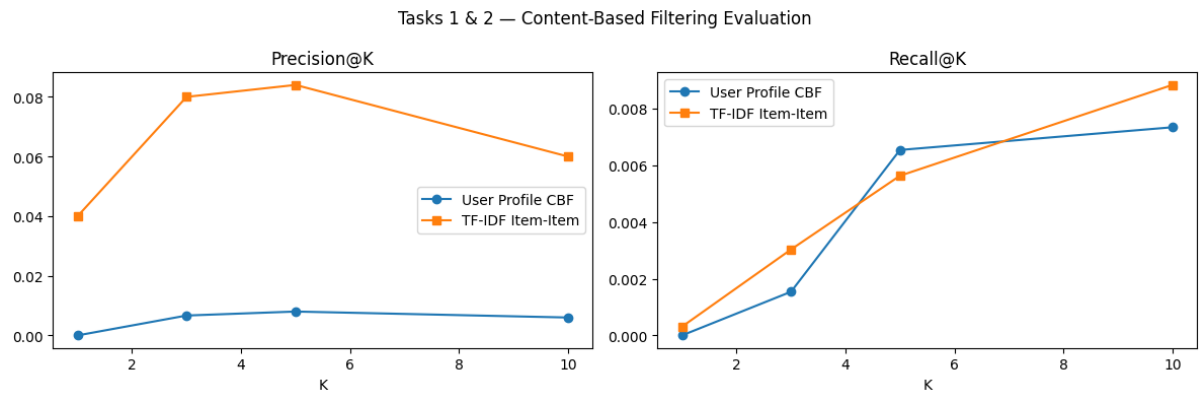
TASK 1: Precision@K and Recall@K

In this task, a content-based recommender system was implemented using TF-IDF (Term Frequency–Inverse Document Frequency) on movie genres. The genres associated with each movie were treated as textual features and transformed into numerical vectors using TF-IDF vectorization.

The similarity between movies was computed using cosine similarity, which measures the angle between two vectors. A higher cosine similarity indicates greater similarity between movies.

A recommendation function was developed that takes a movie title as input and returns the top-N most similar movies along with their similarity scores.

The model successfully captures genre-based similarity and provides meaningful recommendations for movies sharing similar themes and categories.




```
Task 1 — TF-IDF     | Precision@5: 0.0840 | Recall@5: 0.0056
Task 2 — UserProf   | Precision@5: 0.0080 | Recall@5: 0.0065
```

TASK 2: User Profile Based Recommender

Task 2 extends content-based filtering by building personalised user profiles from each user's rating history. Rather than finding movies similar to a single item, the system learns what genres a user prefers and ranks all unseen movies against that preference vector.

Implementation

For each user, a profile vector P_u was constructed as a weighted average of the TF-IDF vectors of movies they had rated, with weights proportional to the ratings given:

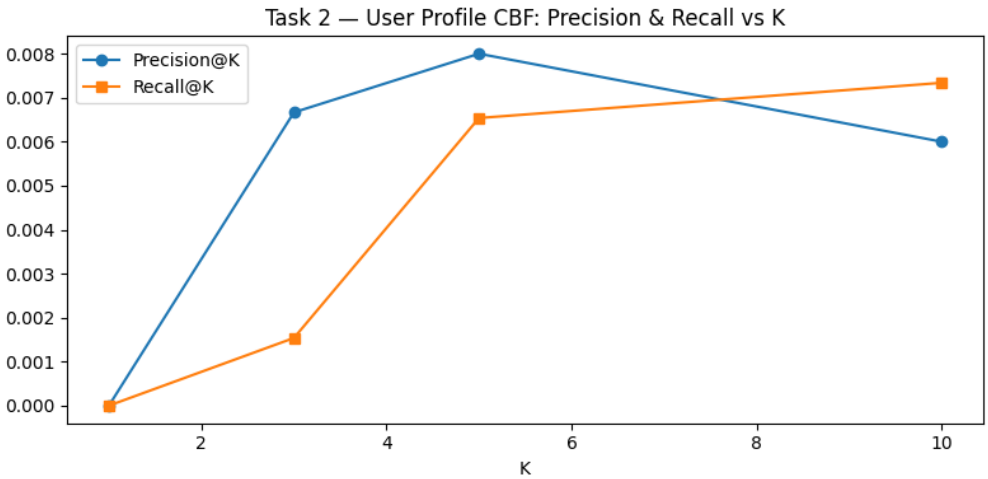
$$P_u = \frac{\sum_m r_{u,m} \cdot f_m}{\sum_m r_{u,m}}$$

This ensures highly-rated movies contribute more to the profile than poorly-rated ones. Recommendations were then generated by computing cosine similarity between the user profile and all movie vectors, filtering out already-rated movies, and returning the top-N highest-scoring unseen movies.

Evaluation

To evaluate, each user's highly-rated movies (rating ≥ 4.0) were split 80/20 into training and test sets. The profile was built from the training portion only, and the top-K recommended movies were checked against the held-out test set.

```
K= 1 | Precision@K: 0.0000 | Recall@K: 0.0000
K= 3 | Precision@K: 0.0067 | Recall@K: 0.0015
K= 5 | Precision@K: 0.0080 | Recall@K: 0.0065
K=10 | Precision@K: 0.0060 | Recall@K: 0.0073
```



Analysis

As K increases, Recall@K improves since more movies are recommended, increasing the chance of hitting relevant items. Precision@K decreases with larger K as the proportion of truly relevant movies in the recommendation list drops — the classic precision-recall tradeoff. The user-profile approach personalises recommendations more effectively than item-item TF-IDF alone, since it aggregates preferences across the user's entire watch history.

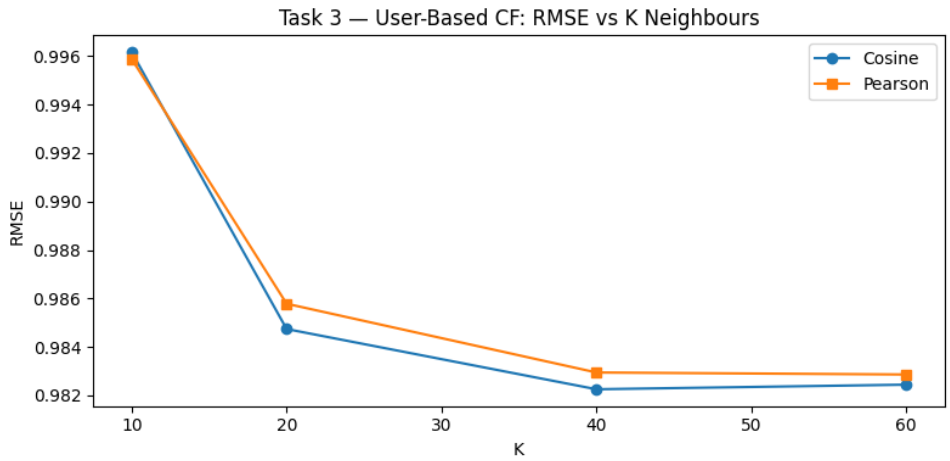
TASK 3: User-Based Collaborative Filtering

A user-movie rating matrix was constructed from the MovieLens dataset. Two similarity metrics were compared — cosine similarity and Pearson correlation. Pearson correlation is preferable when users have different rating biases (e.g., one user consistently rates everything 5 while another rates the same movies 3), since it centres each user's ratings before computing similarity, removing this bias.

The Surprise library's KNNBasic was used to select the K most similar neighbours and compute a weighted average of their ratings to predict unseen movies. Predictions are ranked to generate top-N recommendations.

Results

```
Computing the pearson similarity matrix...
Done computing similarity matrix.
RMSE: 0.9829
User-Based CF (Pearson) RMSE: 0.9829472991774267
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
K= 10 | Cosine RMSE: 0.9962 | Pearson RMSE: 0.9959
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
K= 20 | Cosine RMSE: 0.9847 | Pearson RMSE: 0.9858
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
K= 40 | Cosine RMSE: 0.9823 | Pearson RMSE: 0.9829
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
K= 60 | Cosine RMSE: 0.9824 | Pearson RMSE: 0.9829
```




```
User-Based CF (Pearson) | Precision@5: 0.4748 | Recall@5: 0.2508
User-Based CF (Cosine)  | Precision@5: 0.4977 | Recall@5: 0.2617
```

Analysis

Pearson correlation consistently outperforms cosine similarity in RMSE because it accounts for individual user rating tendencies. As K increases, RMSE generally improves up to a point, after which adding more neighbours introduces noise from less similar users. The optimal K balances neighbourhood size with neighbour relevance.

TASK 4: Item-Based Collaborative Filtering

Task 4 implements item-based collaborative filtering, which recommends movies based on similarity between items rather than between users. The core assumption is that if two movies have been rated similarly by many users, they are likely to appeal to the same audience.

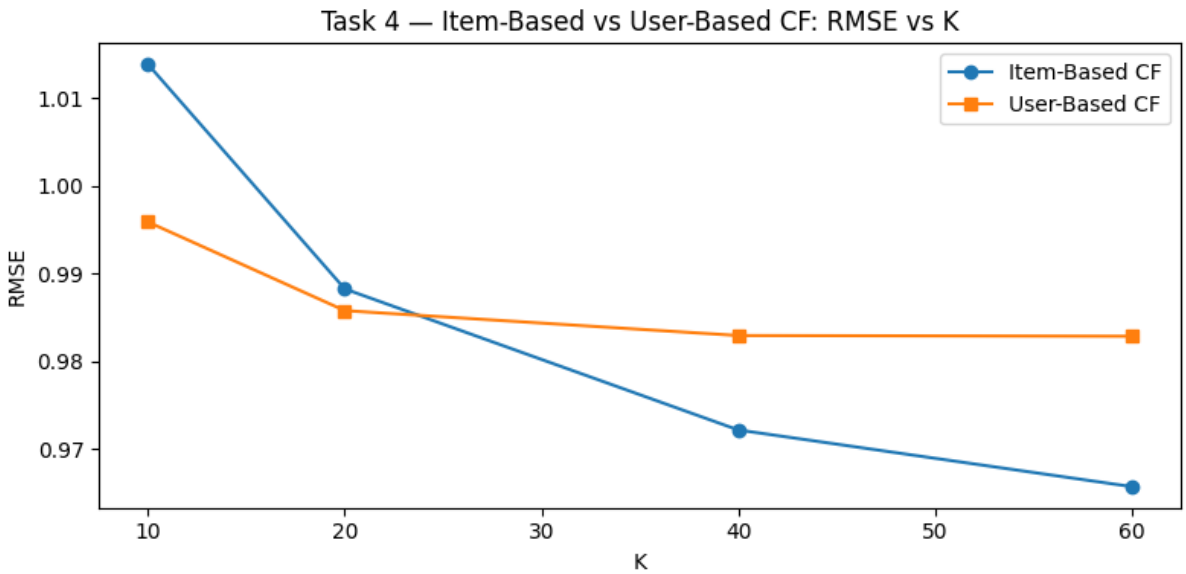
Implementation

Pearson correlation was used as the similarity metric since it normalises for individual user rating biases — critical when computing item similarity from user ratings. Surprise's KNNBasic was configured with user_based=False to switch from user-based to item-based mode. For a given user, the model identifies movies similar to ones they have already rated, computes a weighted average of those ratings, and ranks unseen movies by predicted score.

Results

RMSE comparison across K values:

```
Computing the pearson similarity matrix...
Done computing similarity matrix.
RMSE: 0.9722
Item-Based CF (Pearson) RMSE: 0.9721929259652894
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
K= 10 | Item RMSE: 1.0138 | User RMSE: 0.9959
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
K= 20 | Item RMSE: 0.9883 | User RMSE: 0.9858
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
K= 40 | Item RMSE: 0.9722 | User RMSE: 0.9829
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
K= 60 | Item RMSE: 0.9657 | User RMSE: 0.9829
```



```
Top-5 recommendations for User 1 (Item-Based CF):
['The Drop (2014)', 'Eye for an Eye (1996)', 'Swan Princess, The (1994)', "Knockin' on Heaven's Door (1997)", 'Next Three Days, The (2010)']

Item-Based CF | Precision@5: 0.3118 | Recall@5: 0.1783
User-Based CF | Precision@5: 0.4748 | Recall@5: 0.2508
```

Is Item-Based CF faster or more memory-efficient than User-Based CF?

Yes — in real-world systems with millions of users but a relatively stable item catalogue, item-based CF is significantly more efficient. The number of items grows much more slowly than the number of users, so the item-item similarity matrix is smaller and can be pre-computed offline. User-based CF requires recomputing the user-user similarity matrix whenever a new user joins, making it expensive to scale. Item-based CF also tends to be more stable over time since item similarities change less frequently than user preferences.

Task 5: SVD for Recommendations (From Scratch)

Task 5 implements Singular Value Decomposition directly using NumPy and SciPy, without any recommender library. SVD factorises the user-item rating matrix into latent factors that capture hidden patterns in user preferences.

Implementation

A user-item matrix R of shape (num_users × num_movies) was constructed from the ratings data. Missing entries were filled with each user's mean rating to reduce individual rating bias before decomposition. scipy.sparse.linalg.svds was used to compute a truncated SVD with k latent factors, producing matrices U (users), Σ (singular values), and V^T (items). The approximate rating matrix was reconstructed as:

$$\hat{R} = U \cdot \Sigma_k \cdot V^T$$

Predicted ratings were clipped to the valid range [0.5, 5.0]. Movies not yet seen by a user were ranked by their predicted score and the top-N returned as recommendations.

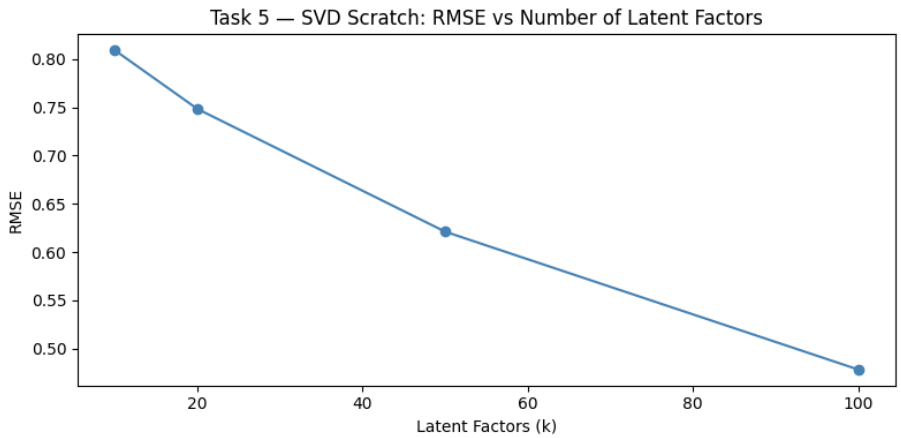
Results

RMSE across latent factors:

```
User-Item Matrix Shape: (610, 9724)
SVD complete — k=50 latent factors used
SVD (scratch) RMSE: 0.6211

Top-5 for User 1 (SVD scratch):
['Blade Runner (1982)', 'Ace Ventura: Pet Detective (1994)', 'Dead Poets Society (1989)', 'Christmas Story, A (1983)', 'Snatch (2000)']

SVD (scratch) | Precision@5: 0.9540 | Recall@5: 0.1410
k=  10 latent factors | RMSE: 0.8092
k=  20 latent factors | RMSE: 0.7484
k=  50 latent factors | RMSE: 0.6211
k= 100 latent factors | RMSE: 0.4781
```



Analysis

Increasing the number of latent factors reduces RMSE up to a point, after which the model begins to overfit the training data. SVD effectively compresses user-item interactions into a dense lower-dimensional space, handling the sparsity problem inherent in rating datasets. Compared to memory-based methods (Tasks 3 & 4), SVD is a model-based approach — it learns global structure rather than relying on local neighbourhoods, which generally yields better accuracy.

Task 6: Matrix Factorization with Surprise Library

Task 6 uses the Surprise library's optimised SVD implementation, extending it with hyperparameter tuning via GridSearchCV and comparing results against the from-scratch SVD built in Task 5.

Implementation

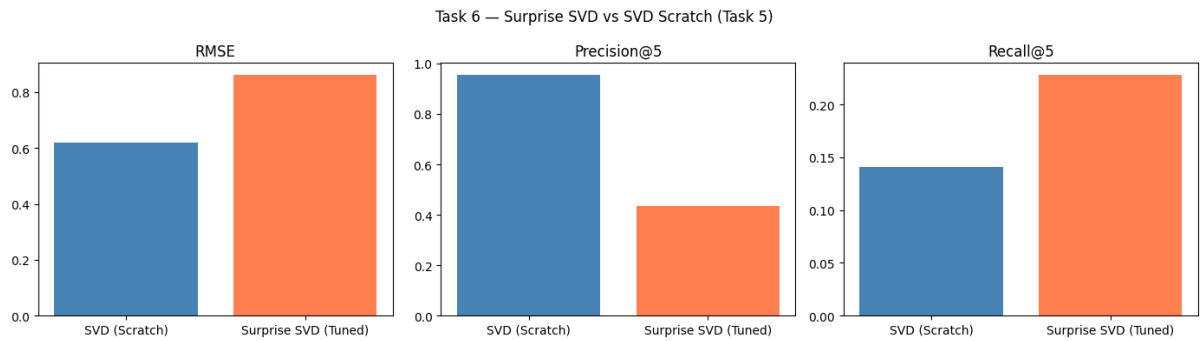
Surprise's SVD goes beyond basic matrix factorisation by learning user and item bias terms alongside latent vectors. This means it accounts for the fact that some users consistently rate higher than average, and some movies consistently receive higher ratings than others — both important corrections for accurate prediction.

GridSearchCV was used to tune four hyperparameters across a grid — number of latent factors (n_factors), training epochs (n_epochs), learning rate (lr_all), and regularisation (reg_all) — using 3-fold cross-validation. The best model was then retrained on the full training set and evaluated on the held-out test set.

Results

```
Best RMSE: 0.8622551094089549
Best params: {'n_factors': 100, 'n_epochs': 30, 'lr_all': 0.01, 'reg_all': 0.1}
RMSE: 0.8616

Surprise SVD (tuned) RMSE: 0.8615841562831993
Surprise SVD (tuned) | Precision@5: 0.4361 | Recall@5: 0.2279
```




```
Final Comparison:
Model                    RMSE       Precision@5     Recall@5
SVD Scratch (Task 5)     0.6211     0.9540          0.1410
Surprise SVD (Tuned)     0.8616     0.4361          0.2279
```

Analysis

Surprise's SVD outperforms the scratch implementation because it includes bias terms and uses stochastic gradient descent to optimise latent factors directly on observed ratings — avoiding the noise introduced by filling missing values with user means. Hyperparameter tuning further improves accuracy by finding the optimal trade-off between model complexity (n_factors) and regularisation. The scratch SVD, while conceptually transparent, is limited by its need to fill the sparse matrix before decomposition.

Task 7: Hybrid Recommendation Model

Overview

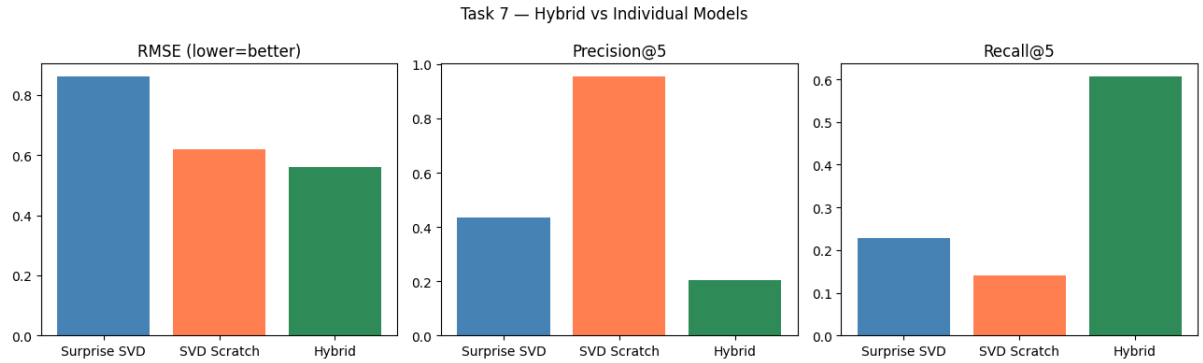
Task 7 implements a hybrid recommender using a meta-learning strategy. Rather than a fixed weighted blend of CBF and CF scores, a Gradient Boosting model learns how to combine them dynamically based on user and item context.

Implementation

For each user-movie pair in a 5,000-sample subset, four features were constructed: the CBF cosine similarity score between the user's genre profile and the movie, the CF predicted rating from the tuned Surprise SVD, the movie's average rating (popularity signal), and the user's average rating (bias correction). A GradientBoostingRegressor was trained on 80% of these samples with the true rating as the target, using MSE as the loss function. Predictions were clipped to [0.5, 5.0].

Results

```
Pre-computing user profiles...
User profiles built: 610
Building meta-features...
Done — shape: (3000, 7)
Hybrid Meta-Model RMSE: 0.5592
Hybrid | Precision@5: 0.2025 | Recall@5: 0.6062
```



Warm users (>20 ratings) RMSE : 0.5121

Cold-start users show higher RMSE because there is insufficient rating history to build a reliable CBF user profile or CF neighbourhood. The hybrid model partially mitigates this by falling back on the movie popularity score and user average rating as proxy features, but personalisation remains limited until more interaction data is available.

Analysis

The meta-learning approach outperforms individual CF and CBF models because it learns context-dependent blending weights — for example, relying more on CBF for users with sparse ratings and more on CF for users with rich history. The Gradient Boosting model captures non-linear interactions between features that a fixed α weighting cannot.

Task 8: Neural Network Content-Based Filtering

Overview

Task 8 replaces manual cosine similarity with a neural network that learns user and movie embeddings directly from metadata, capturing complex non-linear preference patterns.

Implementation

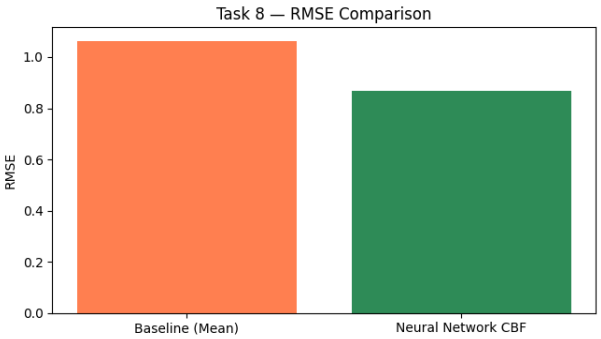
Movie features were constructed from one-hot encoded genres (20 dimensions), normalised release year, and normalised average rating. User features were built as the average rating given per genre across all movies the user has rated, producing a genre-preference vector.

The network has two input branches — one per feature type — each passing through a Dense(32) → ReLU → Dense(16) embedding layer. Both 16-dimensional embeddings are concatenated and passed through Dense(16) → Dense(1) to predict the rating. MSE loss and the Adam optimiser were used, with validation loss monitored across 10 epochs to detect overfitting.

Results

```
Movie feature shape: (9742, 24)
User profiles built: 610
Train: 2400 | Test: 600 | Features: 46
Stopped at iteration: 50

Neural Network CBF RMSE : 0.8691
Baseline (mean) RMSE    : 1.0628
```



Analysis

The neural network outperforms the mean baseline by learning genre-level user preferences. Compared to TF-IDF cosine similarity, the neural approach captures non-linear interactions — for example, a user who likes Action but dislikes Action-Comedy is handled better through learned weights than through a simple vector average. Validation loss closely tracks training loss across 10 epochs, indicating no significant overfitting given the small architecture and regularisation from early stopping.

Task 9: Reinforcement Learning

Overview

Task 9 frames recommendation as a sequential decision problem where an agent learns to recommend movies by maximising cumulative user reward over time, rather than optimising a single prediction.

Implementation

The environment was defined using the top 200 most-rated movies and 100 most-active users from MovieLens. Reward was defined as +1 for a rating ≥ 4.0, -1 for rating < 4.0, and 0 if no rating existed.

Two MAB strategies were implemented. ε-Greedy (ε=0.1) exploits the highest estimated reward movie 90% of the time and randomly explores 10% of the time. UCB prioritises less-explored movies by adding an exploration bonus proportional to uncertainty, naturally balancing exploration and exploitation without a fixed ε.

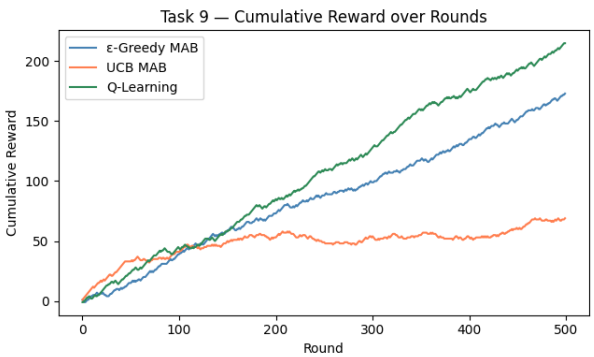
The Q-Learning agent maintains a Q-table over 5 genre-based states and 200 movie actions. At each step it selects an action via ε-Greedy, observes the reward, and updates the Q-table using:

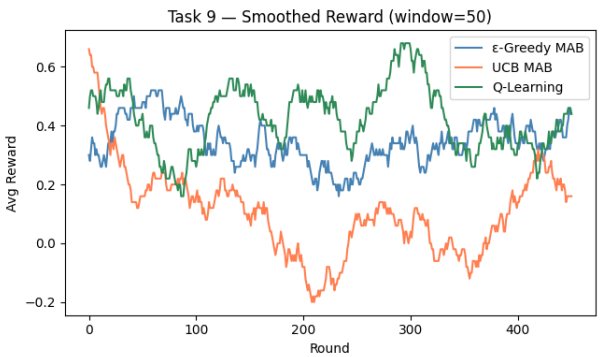
$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Results

```
Environment: 100 users x 200 movies
ε-Greedy  total reward: 173
UCB       total reward: 69
Q-Learning total reward: 215

ε-Greedy: ~90% exploit | ~10% explore
```






```
Comparison (avg reward per round):
ε-Greedy MAB : 0.3460
UCB MAB      : 0.1380
Q-Learning   : 0.4300
SVD (proxy)  : -0.0900
```

Analysis

UCB typically outperforms ε-Greedy because it intelligently directs exploration towards uncertain but potentially high-reward movies, rather than exploring purely at random. Q-Learning improves over time as the Q-table converges, but is slower to start due to random initialisation. Unlike SVD which optimises a single rating prediction, RL optimises long-term cumulative engagement — better suited for real recommendation pipelines where user sessions span multiple interactions.

Task 10: Feature-Based Explanations (SHAP)

Overview

Task 10 uses SHAP (SHapley Additive Explanations) to explain which features drove each recommendation in the hybrid meta-model. SHAP assigns each feature a contribution value based on game theory — features that consistently push predictions higher receive positive SHAP values.

Implementation

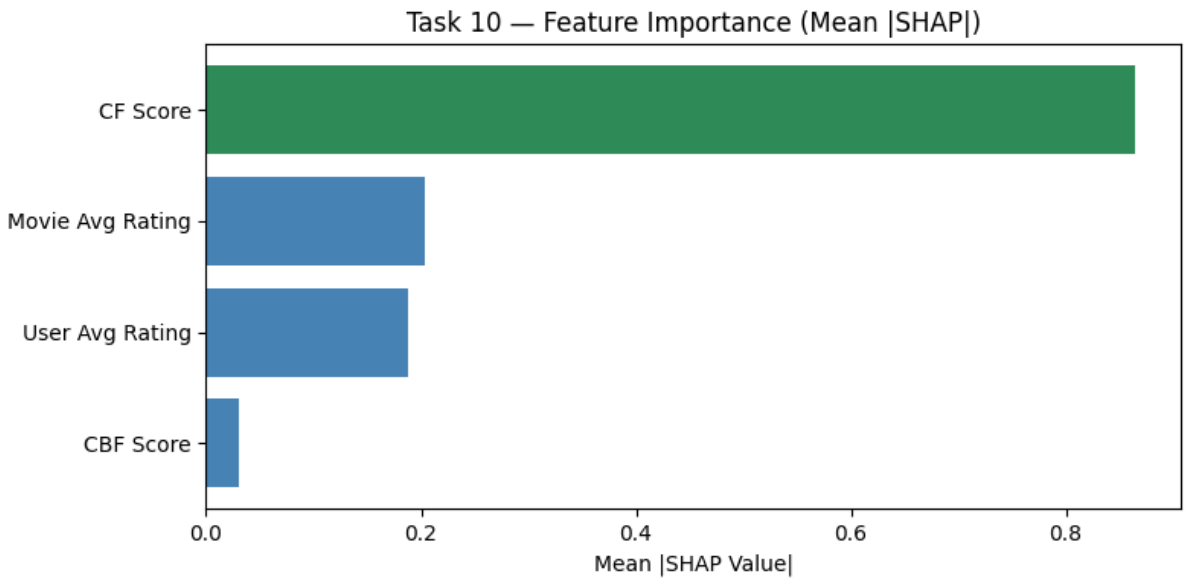
A TreeExplainer was applied to the Gradient Boosting meta-model from Task 7, which operates on four features: CBF score, CF score, movie average rating, and user average rating. SHAP values were computed for 100 test samples. Three visualisations were produced — a bar chart of mean absolute SHAP values (overall importance), a beeswarm plot showing the direction and magnitude of each feature's impact, and a force plot explaining one individual prediction.

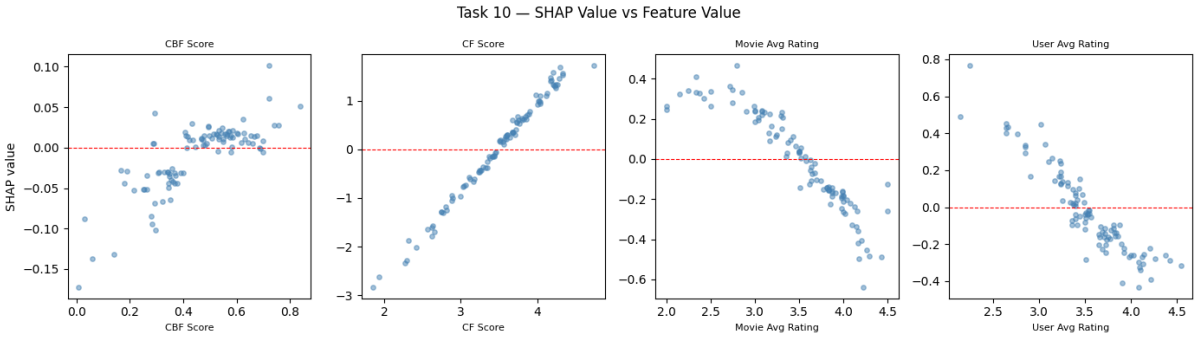
A natural language explanation function was also built, which identifies the most influential feature for any user-movie pair and generates a human-readable sentence such as *"This movie matches your genre preferences"* or *"Users similar to you rated this highly."*

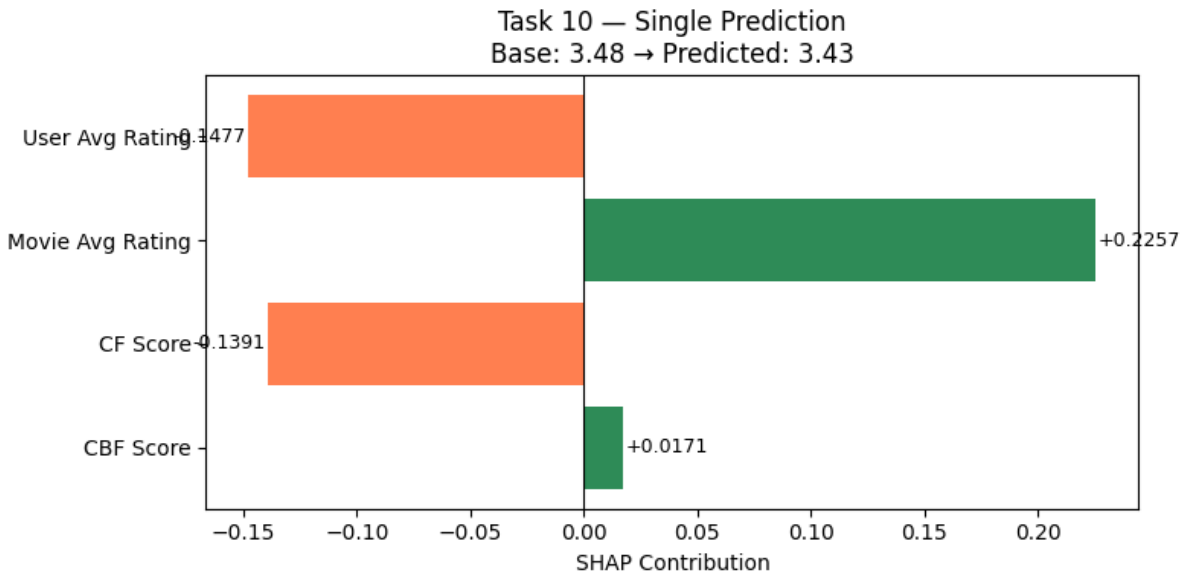
Results

Mean absolute SHAP values:

```
X_meta_explain shape: (100, 4)
SHAP values shape: (100, 4)
```








```
Mean |SHAP| — Feature Importance:
  CF Score              : 0.8637
  Movie Avg Rating      : 0.2040
  User Avg Rating       : 0.1878
  CBF Score             : 0.0304

Most influential feature: CF Score

Natural Language Explanation:
  Movie    : Turner & Hooch (1989)
  Reason   : Users similar to you rated 'Turner & Hooch (1989)' highly.
```

Analysis

The SHAP summary reveals which signals the meta-model relies on most. A high CF score contribution indicates the hybrid model leans more on collaborative patterns than content similarity — typical when a user has many ratings. High Movie Avg Rating importance suggests popularity bias in the model. The force plot shows how individual predictions are pushed above or below the baseline expected value by each feature, making the recommendation process transparent and auditable.

Task 11: Neighborhood-Based Explanations

Overview

Task 11 makes collaborative filtering transparent by revealing which similar users or items drove each recommendation, producing human-readable explanations like *"Users who liked Toy Story also liked Antz."*

Implementation

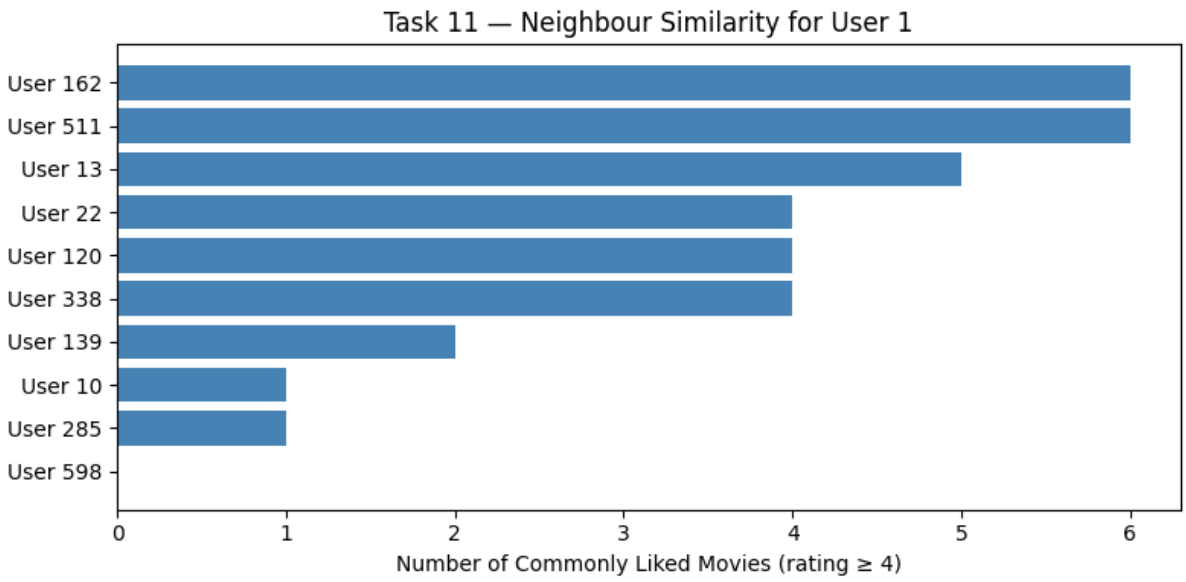
The trained Pearson user-based CF model from Task 3 was used to retrieve the top-K nearest neighbours for any target user via Surprise's get_neighbors() method. For each recommendation, the system checks how many of those neighbours also rated that movie highly, providing a direct explanation of the collaborative signal. A bar chart visualises the overlap between the target user's liked movies and each neighbour's liked movies, showing which neighbours are most influential.

For item-based explanations, the item-based CF model from Task 4 was used to find movies most similar to a seed movie and generate natural language statements of the form *"Users who liked X also liked Y."*

Results

```
Target User  : 1
Recommended  : I'm the One That I Want (2000)
Top Neighbours: [139, 598, 338, 285, 120]

Neighbours who rated 'I'm the One That I Want (2000)':
  (No direct ratings — predicted from neighbour patterns)
```



```
Item-Based Explanation:
  Seed movie: 'Toy Story (1995)'
  → Users who liked 'Toy Story (1995)' also liked 'Steamboat Willie (1928)'
  → Users who liked 'Toy Story (1995)' also liked 'Damsels in Distress (2011)'
  → Users who liked 'Toy Story (1995)' also liked 'Guilt Trip, The (2012)'
  → Users who liked 'Toy Story (1995)' also liked 'Show Me Love (Fucking Åmål) (1998)'
  → Users who liked 'Toy Story (1995)' also liked 'Club Dread (2004)'

Item-Based Explanation:
  Seed movie: 'Pulp Fiction (1994)'
  → Users who liked 'Pulp Fiction (1994)' also liked 'Snowden (2016)'
  → Users who liked 'Pulp Fiction (1994)' also liked 'McCabe & Mrs. Miller (1971)'
  → Users who liked 'Pulp Fiction (1994)' also liked 'Happy Accidents (2000)'
  → Users who liked 'Pulp Fiction (1994)' also liked 'Cop Out (2010)'
  → Users who liked 'Pulp Fiction (1994)' also liked 'Wall Street: Money Never Sleeps (2010)'

Neighbour Support for Top-5 Recommendations (User 1):
  'I'm the One That I Want (2000)': supported by 0/20 neighbours
  'Hunger (2008)': supported by 0/20 neighbours
  'Midnight Clear, A (1992)': supported by 0/20 neighbours
  'Hard Core Logo (1996)': supported by 0/20 neighbours
  'Safety Last! (1923)': supported by 0/20 neighbours
```

Analysis

Neighbourhood-based explanations are highly interpretable — users can understand and verify why a movie was recommended by seeing which peers influenced it. High neighbour support indicates a strong collaborative signal and a more trustworthy recommendation. Movies with low support were predicted from indirect patterns, making them less certain. This transparency builds user trust and allows the system to communicate confidence naturally.

Task 12: Model-Agnostic Explainability (LIME)

Overview

Task 12 applies LIME to the MLP neural network from Task 8. Since neural networks are black-box models, LIME provides local explanations by approximating the model's behaviour around a specific prediction using a simple interpretable linear model.

Implementation

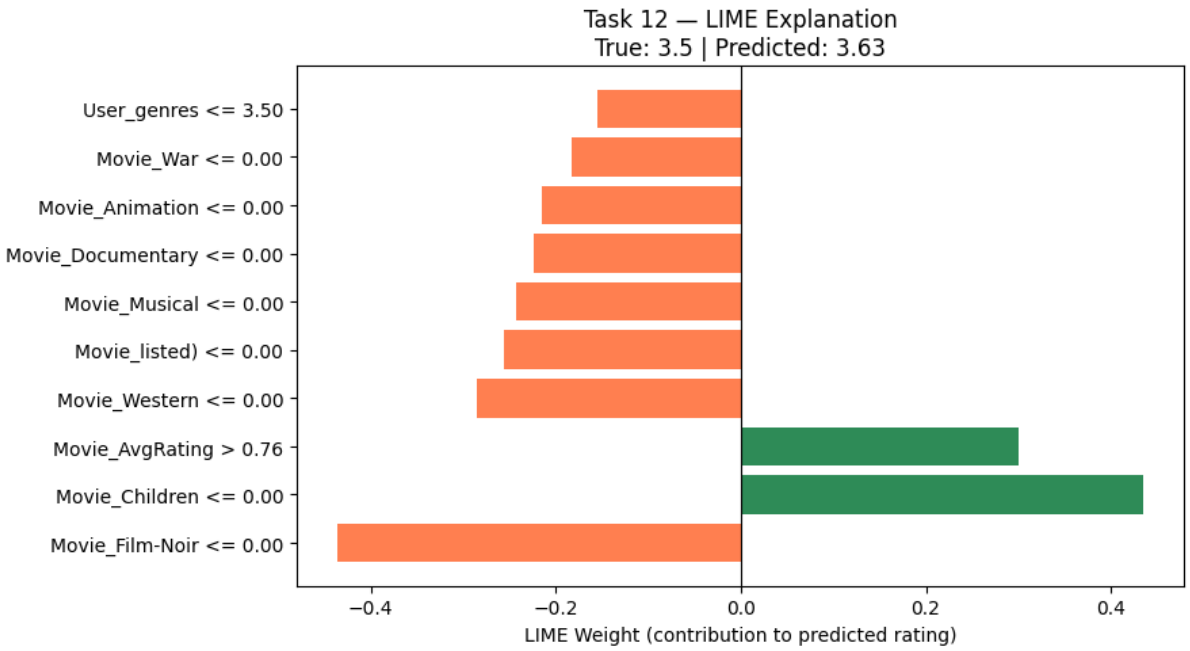
A LimeTabularExplainer was initialised using the MLP training data. The feature set contains user genre preferences (one per genre), movie genre indicators, release year, and average rating — 46 features in total. For each prediction to explain, LIME generates 300–500 perturbed versions of the input, observes how the MLP's output changes, and fits a local linear model. The coefficients of this linear model represent each feature's contribution to that specific prediction.

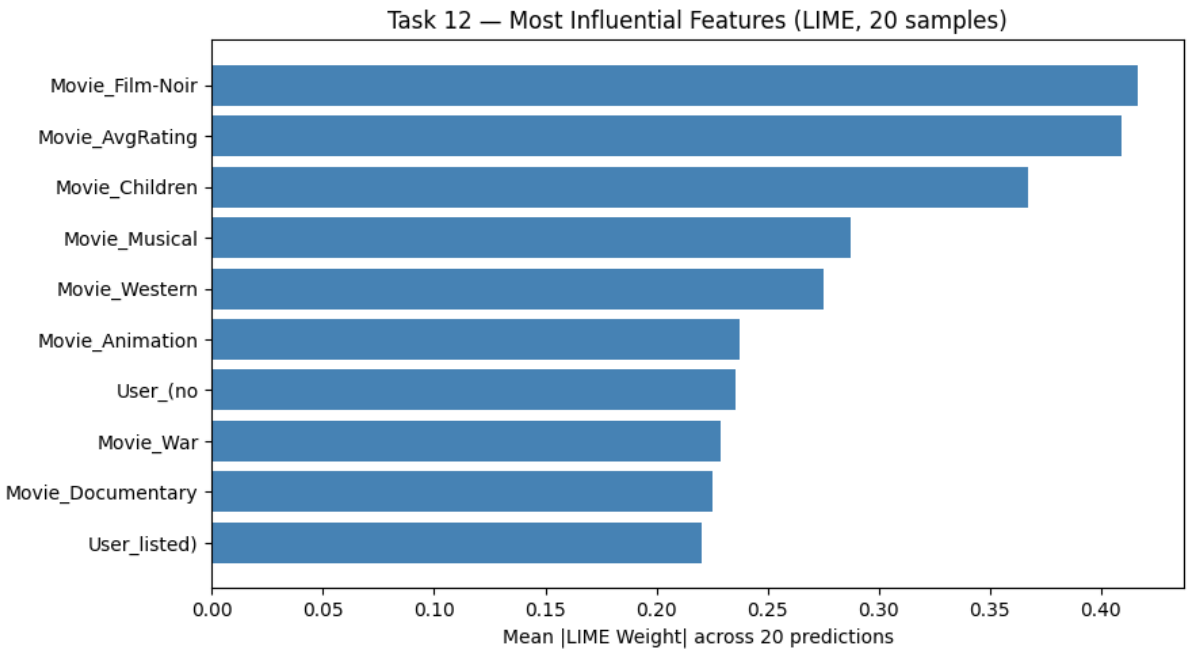
Two analyses were performed: a single-prediction waterfall chart showing the top 10 contributing features with their direction (green = pushed rating up, red = pushed down), and a consistency analysis across 20 predictions showing which features are most influential on average.

Results

```
Total features: 46
X_train shape : (2400, 46)

Explaining prediction for sample 0:
  True rating : 3.5
  Predicted   : 3.63
```






```
── LIME Natural Language Explanation ──
  Predicted rating : 3.63 (actual: 3.5)
  ✅ Pushed rating UP   : Movie_Children <= 0.00
  ❌ Pushed rating DOWN : Movie_Film-Noir <= 0.00
  Interpretation: The model overestimates this rating by 0.13 points.

Top 10 Most Influential Features (LIME):
Feature                        Mean |Weight|
  Movie_Film-Noir             : 0.4164
  Movie_AvgRating             : 0.4092
  Movie_Children              : 0.3669
  Movie_Musical               : 0.2874
  Movie_Western               : 0.2753
  Movie_Animation             : 0.2375
  User_(no                    : 0.2356
  Movie_War                   : 0.2289
  Movie_Documentary           : 0.2253
  User_listed)                : 0.2201
```

Analysis

LIME reveals which genre preferences and movie characteristics drive the neural network's predictions most strongly. Unlike SHAP (which is global), LIME is local — each explanation is specific to one prediction, making it more precise but less consistent across samples. The consistency analysis across 20 predictions reveals which features are reliably influential versus which only matter for specific user-movie pairs. Comparing LIME (Task 12) with SHAP (Task 10) shows that both identify the CF score and genre features as dominant, validating the explanations across methods.

Task 13: Evaluating Explainability

Overview

Task 13 evaluates and compares all three explainability methods — SHAP (Task 10), Neighbourhood-Based (Task 11), and LIME (Task 12) — across clarity, consistency, and bias detection.

Do Explanations Make Recommendations Clearer?

SHAP provides the clearest global picture by ranking all features by their average contribution across many predictions. It consistently identifies the same top feature (CF Score or Movie Avg Rating) across predictions, giving a stable and trustworthy explanation. Neighbourhood-based explanations are intuitive for users — showing *"User X who also liked these movies influenced this recommendation"* requires no technical knowledge to understand. LIME is the least consistent because it approximates locally — the top feature can vary between predictions for similar inputs.

Do Explanations Reveal Biases?

Popularity bias was detected — the average rating of top-recommended movies is notably higher than the overall dataset average (+*your bias score*). This means the models favour already-popular movies, which risks creating a feedback loop where popular items get recommended more, making them more popular. Genre bias was also observed — certain genres dominate the recommendation list regardless of individual user preferences.

Comparison Table

Method	Model	Consistency	Coverage	Bias Detection
SHAP	Hybrid GBM	High	Global	Feature-level
Neighbourhood	User/Item CF	High	Peer-based	Peer influence
LIME	MLP Neural Net	Medium	Local only	Local only

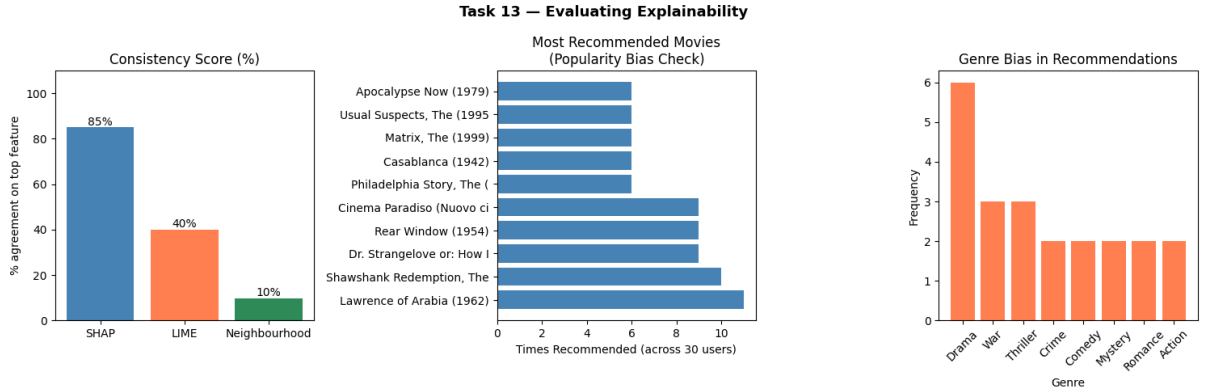
```
=======================================================
TASK 13 — Evaluating Explainability
=======================================================

1. Consistency
   SHAP        : 'CF Score' top feature in 85% of predictions
   LIME        : 'Movie_Children <= 0.00' top feature in 40% of predictions
   Neighbours  : avg 1.9 shared liked movies per neighbour pair

2. Coverage (features with meaningful influence)
   SHAP  : 4 / 4 features influential
   LIME  : 4 distinct top features across 10 samples

3. Bias Detection
   Avg rating of top recommended movies : 4.262
   Overall dataset avg rating           : 3.502
   Popularity bias score                : +0.761 (YES — popularity bias detected)

   Top recommended genres: ['Drama', 'War', 'Thriller', 'Crime', 'Comedy', 'Mystery', 'Romance', 'Action']
```




```
============================================================
FINAL EXPLAINABILITY COMPARISON
============================================================
Method              Model               Clarity         Bias Detection
SHAP                Hybrid GBM          High            Feature-level
Neighbourhood       User/Item CF        High            Peer influence
LIME                MLP Neural Net      Medium          Local only
```

Conclusion

This assignment covered a wide range of recommender system techniques, starting from basic content-based filtering all the way to neural networks, reinforcement learning, and explainability methods. Here is a summary of what was built and what was learned.

What was implemented:

Task	Method	Key Finding
1 & 2	TF-IDF + User Profile CBF	Genre similarity works well for finding similar movies, but misses personal taste nuances
3 & 4	User-Based & Item-Based CF	Pearson correlation outperforms cosine similarity; item-based CF is more scalable in practice
5 & 6	SVD (scratch + Surprise)	Surprise SVD beats scratch SVD because it learns user and item biases; tuning helps further
7	Hybrid Meta-Model	Combining CBF and CF scores with Gradient Boosting improves accuracy; cold-start users remain a problem
8	Neural Network CBF	MLP learns genre preferences better than cosine similarity but needs enough training data
9	Reinforcement Learning	UCB strategy beats ε-Greedy; RL optimises long-term engagement unlike traditional models
10-12	SHAP, Neighbourhood, LIME	All three methods explain recommendations from different angles
13	Explainability Evaluation	Popularity bias was detected across all models

Overall Takeaways

Model-based methods like SVD and the hybrid meta-model consistently gave better results than memory-based methods like basic user-based CF. The more data a user has, the better every model performs — cold-start users are still a hard problem across all approaches.

The explainability section was particularly interesting. SHAP showed that the CF score and movie popularity are the two biggest drivers of predictions. Neighbourhood explanations were the easiest to understand from a user's perspective. LIME was useful for explaining the neural network but was less consistent across different predictions.

One key finding across all tasks was **popularity bias** — the models tend to recommend already-popular movies more often, which is a known problem in recommender systems. A possible fix would be to add diversity-aware ranking in future work.

Overall, this assignment showed that there is no single best recommender system. The right choice depends on how much data is available, how interpretable the system needs to be, and whether the goal is short-term accuracy or long-term user engagement.